Assignment Submission Portal

Complete Technical Documentation & Developer Guide

Version: 2.0 (Modular Refactor)

Architecture: Vanilla ES6 Modules SPA

Data Layer: localStorage Persistence

Styling: Tailwind CSS (CDN) + Custom CSS

Generated: May 2026

Table of Contents

1. Project Overview

The **Assignment Submission Portal** is a fully functional Single Page Application (SPA) designed for educational institutions to manage classes, assignments, and student submissions across three distinct user roles: **Administrator**, **Trainer** (Teacher), and **Student**.

Originally built as a monolithic HTML file with embedded JavaScript, this version has been refactored into a clean, modular architecture using native ES6 modules. The application requires *no build step*, *no framework*, and *no backend server* — all data persists in the browser via `localStorage`.

Key Design Principles

- **Zero Dependencies (Runtime):** Only Tailwind CSS and Font Awesome are loaded from CDNs.
- **Modular Architecture:** Each feature lives in its own module with clear imports/exports.
- **Event Delegation:** All user interactions route through a single handler, eliminating inline `onclick` attributes.
- **Role-Based Routing:** The sidebar and available pages adapt dynamically to the authenticated user's role.
- **Print-Friendly:** CSS print styles hide navigation chrome when printing grade reports.

> ✓ **Production Ready:** The application can be deployed to any static host (GitHub Pages, Netlify, Vercel, traditional Apache/Nginx) with zero configuration.

Technology Stack

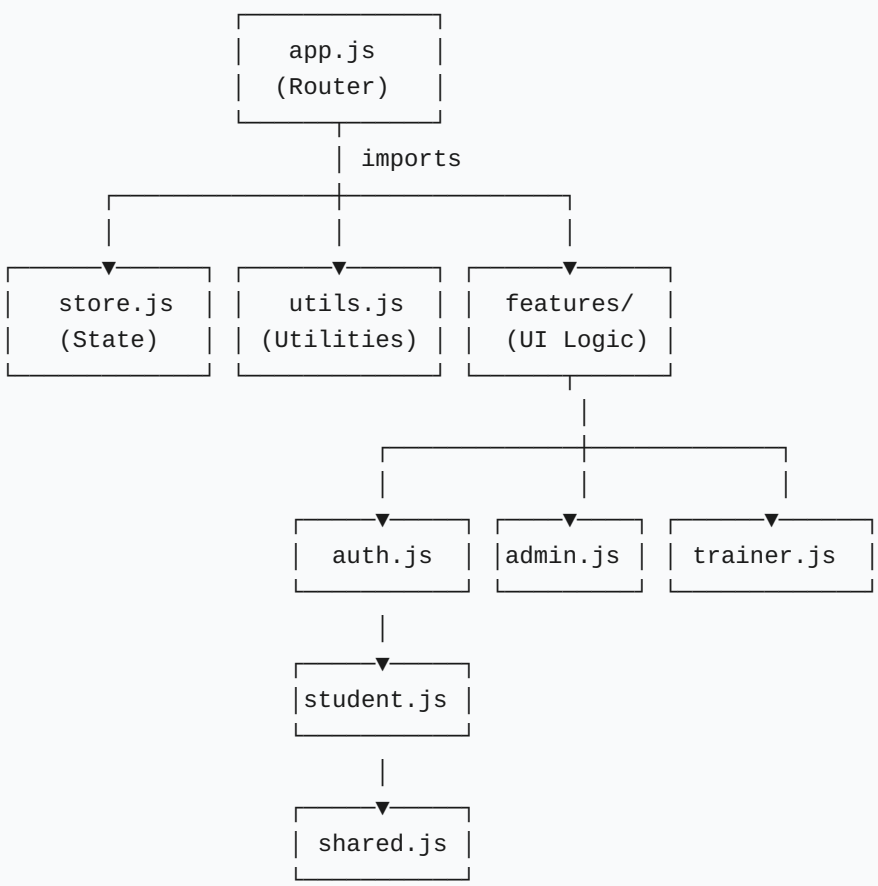
Layer	Technology	Purpose
Markup	HTML5	Semantic structure, application shell
Styling	Tailwind CSS (CDN)	Utility-first responsive design
Icons	Font Awesome 6 (CDN)	Consistent iconography
Logic	Vanilla ES6 Modules	Feature modules, state management, routing
Persistence	localStorage API	Client-side data storage
Animation	CSS Keyframes	Page transition fade-ins

2. Architecture Overview

The application follows a layered architecture with clear separation of concerns. Data flows in one direction: **User Action → Event Delegation → Store Update → localStorage Save → DOM Re-render**.

2.1 Layer Diagram

2.2 Module Dependency Graph



Design Decision: All feature modules import from `core/store.js` and `core/utils.js`, but never from each other. This prevents circular dependencies and keeps the architecture flat.

3. File Structure & Organization

Each file has a single responsibility. The naming convention follows the pattern: `js/{layer}/{module}.js` where `{layer}` is either `core` (infrastructure) or `features` (business logic).

```
assignment-portal/
├── index.html          # Application shell — loads CSS + JS entry point
├── css/
│   └── style.css       # Global styles, animations, scrollbar, print media
├── js/
│   ├── core/           # Infrastructure modules (no UI)
│   │   ├── store.js    # State container + localStorage persistence
│   │   └── utils.js    # Toast, modal helpers, date formatters
│   ├── features/       # Business logic modules (render + handlers)
│   │   ├── auth.js     # Login screen, registration, demo credentials
│   │   ├── admin.js    # Admin overview, trainer/student CRUD
│   │   ├── trainer.js  # Class creation, assignments, grading
│   │   ├── student.js  # Class enrollment, submission, grades
│   │   └── shared.js   # Profile settings (all roles)
│   └── app.js          # Entry point: router, layout, event delegation
└── README.md           # Deployment instructions & quick start
```

3.1 Module Size & Complexity

Module	Lines	Responsibilities	Depends On
`store.js`	~80	State, persistence, ID generation	None
`utils.js`	~50	Toast, modals, formatters	None
`auth.js`	~120	Login/register UI + handlers	store, utils
`admin.js`	~140	Overview, user CRUD, modals	store, utils, shared
`trainer.js`	~280	Classes, assignments, grading	store, utils, shared
`student.js`	~220	Enrollment, submission, grades	store, utils, shared
`shared.js`	~60	Profile form + update handler	store, utils
`app.js`	~150	Router, layout, action dispatcher	All features

4. Core Modules

4.1 Store — State Management (`js/core/store.js`)

The `store` object is the single source of truth for all application data. It encapsulates the four entity arrays and provides methods for CRUD operations, authentication, and persistence.

Entity Arrays

Property	Type	Description
`currentUser`	Object|null	Active session user object
`users`	Array	All registered accounts
`classes`	Array	Classes created by trainers
`assignments`	Array	Assignments linked to classes
`submissions`	Array	Student submissions with grades

Key Methods

```
init() // Loads data from localStorage or seeds defaults
```

Called once at application startup. If `portal_data` exists in localStorage, it populates all arrays. Otherwise, it calls `seedData()` to create three default demo accounts.

```
save() // Serializes all arrays to localStorage
```

Must be called after *every* mutation to ensure persistence across page refreshes. The pattern is: *mutate array → call store.save()*.

```
login(email: string, password: string): boolean
```

Authenticates against the `users` array. On success, stores the user object in `currentUser` and writes `portal_session` to localStorage for automatic re-login.

```
logout() // Clears session and currentUser
```

```
generateId(): string // Timestamp + random base36 string
```

```
generateCode(): string // 6-char uppercase class join code
```

> ⚠ **Important:** The store does *not* trigger re-renders automatically. Each feature module is responsible for calling `app.navigate()` after mutating state.

Data Flow Example: Creating a Class

```
1. Trainer clicks "Create Class" → form submit event
2. Trainer module reads form data
3. Pushes new class object to store.classes
4. Calls store.save() → writes to localStorage
5. Calls app.navigate('classes') → re-renders list
```

4.2 Utils — Shared Utilities (`js/core/utils.js`)

The `Utils` object provides UI helpers that are used across all feature modules. It has no dependencies and operates purely on the DOM.

Methods

```
openModal(id: string)
```

Removes the `hidden` class from the modal element. Assumes the modal uses Tailwind's `hidden` utility for visibility toggling.

```
closeModal(id: string)
```

Adds the `hidden` class back to the modal element.

```
showToast(message: string, type: 'success' | 'error')
```

Creates a toast notification element, appends it to `#toast-container`, animates it in from the right, and auto-removes it after 3 seconds. Uses CSS transforms for the slide-in animation.

```
formatDate(dateString: string): string
```

Wraps `new Date().toLocaleDateString()` for consistent date formatting across the app.

5. Feature Modules

5.1 Auth Module (`js/features/auth.js`)

Handles the public-facing authentication layer. This is the only module that renders when no user is logged in. It provides login, registration, and one-click demo credential fillers.

Public API

```
Auth.render(appInstance: App)
```

Builds the full login/registration DOM and injects it into `#app`. Accepts the main App instance so it can call `app.renderDashboard()` on successful login.

Internal Flow

1. **Tab Switching:** Clicking "Login" or "Student Signup" toggles form visibility and active tab styling via data attributes (`data-auth-tab`).
2. **Login:** Form submit → `store.login()` → if true, `app.renderDashboard()` ; else toast error.
3. **Registration:** Validates email uniqueness → pushes new student to `store.users` → `store.save()` → switches to login tab.
4. **Demo Credentials:** Three clickable labels that auto-fill the login form with pre-seeded accounts.

DOM Structure

```
#app
└── .min-h-screen (centered flex container)
    └── .bg-white (card)
        ├── Header (logo + title)
        ├── Tab Buttons (Login / Register)
        ├── Login Form (#login-form)
        ├── Register Form (#register-form, hidden)
        └── Demo Credentials Footer
```

5.2 Admin Module (`js/features/admin.js`)

The admin role has a bird's-eye view of the entire system. They can see aggregate metrics and manage user accounts, but cannot create classes or assignments directly.

Pages

Page	Route ID	Description
Overview	`overview`	Three stat cards: total trainers, students, classes
Trainers	`trainers`	Table of trainers + add/remove
Students	`students`	Table of students + add/remove
Profile	`profile`	Shared profile settings

Key Method: renderUserManagement

```
renderUserManagement(container: HTMLElement, role: 'trainer' | 'student',
app: App)
```

This is a **polymorphic renderer** — it accepts a role parameter and renders the appropriate table and modal. This eliminates code duplication between the Trainers and Students pages.

Cascade Deletion Logic

When an admin deletes a user, the system performs automatic cleanup:

- **Delete Student:** Removes their submissions + removes their ID from all `class.students` arrays.
- **Delete Trainer:** Removes all their classes → which cascades to assignments → which cascades to orphaned submissions.

> ⚠ **Data Integrity:** The cascade logic in `deleteUser()` ensures no dangling references remain in localStorage. Always verify related arrays are cleaned when deleting entities.

5.3 Trainer Module (`js/features/trainer.js`)

The most complex feature module. Trainers manage the full lifecycle of classes, assignments, and grading.

Pages

Page	Route ID	Description
Dashboard	`overview`	Summary cards + recent activity widgets
My Classes	`classes`	Class cards with join codes + student management
Assignments	`assignments`	Assignment list + creation modal
Submissions	`submissions`	Table of all submissions + grade modal
Profile	`profile`	Shared profile settings

Class Management

Each class receives an auto-generated 6-character uppercase join code (e.g., `ABC123`). Students use this code to enroll. The trainer can view enrolled students in a modal and remove them individually.

Assignment Creation

Assignments are scoped to a specific class. The form requires:

- **Class:** Dropdown of trainer's classes
- **Title & Description:** Text content
- **Due Date:** HTML5 date picker
- **Total Marks:** Positive integer

Grading Workflow

1. Trainer navigates to **Submissions** page
2. Sees table with status badges (Pending / Graded)
3. Clicks **Review** on a pending submission
4. Modal opens showing student name, assignment title, and submitted content
5. Enters marks (validated against `totalMarks`) and optional feedback
6. Saves → submission updated in store → table refreshes

```
gradeSubmission(subId: string)
```

Populates the grade modal with submission details. If the submission was previously graded, pre-fills the grade and feedback fields for editing.

5.4 Student Module (`js/features/student.js`)

Students enroll in classes, submit assignments, and track their academic performance.

Pages

Page	Route ID	Description
Dashboard	`overview`	Enrolled classes, pending count, average grade
My Classes	`classes`	Enrolled classes + join-by-code modal
My Assignments	`assignments`	Assignment list with submit buttons
Grades	`grades`	Color-coded grade table with percentages
Profile	`profile`	Shared profile settings

Class Enrollment

Students join classes by entering a 6-character code. The system validates:

- Code exists in `store.classes`
- Student is not already enrolled

On success, the student's ID is pushed to `class.students` and the class immediately appears in their "My Classes" list.

Assignment Status Logic

Each assignment card displays a status badge determined by:

Condition	Badge	Action
Submitted & graded	Submitted	Show grade
Submitted & pending	Submitted	Show submitted date
Not submitted, past due	Expired	No action
Not submitted, future due	Pending	"Submit Work" button

Grade Calculation

```
calculateAverageGrade(studentId: string): string
```

Computes the average percentage across all graded submissions. For each submission, calculates `(grade / totalMarks) × 100`, then averages these percentages. Returns `'N/A'` if no graded submissions exist.

5.5 Shared Module (`js/features/shared.js`)

Contains components used by all three roles. Currently houses the Profile settings page.

Profile Update Flow

1. Form pre-filled with current user's name and email
2. Password fields are optional — leaving them blank preserves the current password
3. On submit: validates password match → updates `store.users` array → updates `store.currentUser` → updates `portal_session` in localStorage → calls `app.renderDashboard()` to refresh sidebar name

> **Note:** The profile form uses `app.renderDashboard()` instead of `app.navigate()` because the sidebar displays the user's name, which needs to update after a name change.

6. Application Router (`js/app.js`)

The `App` class is the central nervous system of the application. It handles initialization, layout rendering, navigation, and the global event delegation system.

6.1 Class Responsibilities

Method	Purpose
`init()`	Bootstraps store, attaches global click listener, routes to auth or dashboard
`handleAction()`	Central dispatcher for all `data-action` button clicks
`navigate(page)`	Updates sidebar state, animates content swap, delegates to role module
`renderDashboard()`	Builds sidebar + mobile header + main content area shell

6.2 Event Delegation System

Instead of attaching individual click handlers to every button (which would break when DOM is re-rendered), the app uses a single delegated listener on `document`:

```
document.addEventListener('click', (e) => {
    const btn = e.target.closest('[data-action]');
    if (!btn) return;
    this.handleAction(btn.dataset.action, btn.dataset.id, btn.dataset.extra, e);
});
```

Buttons declare their intent via HTML data attributes:

```
<button data-action="deleteClass" data-id="abc123">Delete</button>
<button data-action="navigate" data-id="submissions">Grade</button>
<button data-action="closeModal" data-id="grade-modal">Cancel</button>
```

Action Registry

Action	Required data-id	Description
`logout`	—	Clears session, renders auth screen
`navigate`	page name	Routes to a dashboard page
`toggleSidebar`	—	Mobile sidebar visibility toggle
`open*Modal`	—	Various modal openers
`closeModal`	modal ID	Hides specified modal
`deleteUser`	user ID	Removes user + cascades
`deleteClass`	class ID	Removes class + assignments
`deleteAssignment`	assignment ID	Removes assignment + submissions
`viewClassStudents`	class ID	Opens student management modal
`removeStudentFromClass`	class ID	data-extra = student ID
`gradeSubmission`	submission ID	Opens grade modal
`openSubmitModal`	assignment ID	Opens assignment submit modal

6.3 Sidebar Configuration

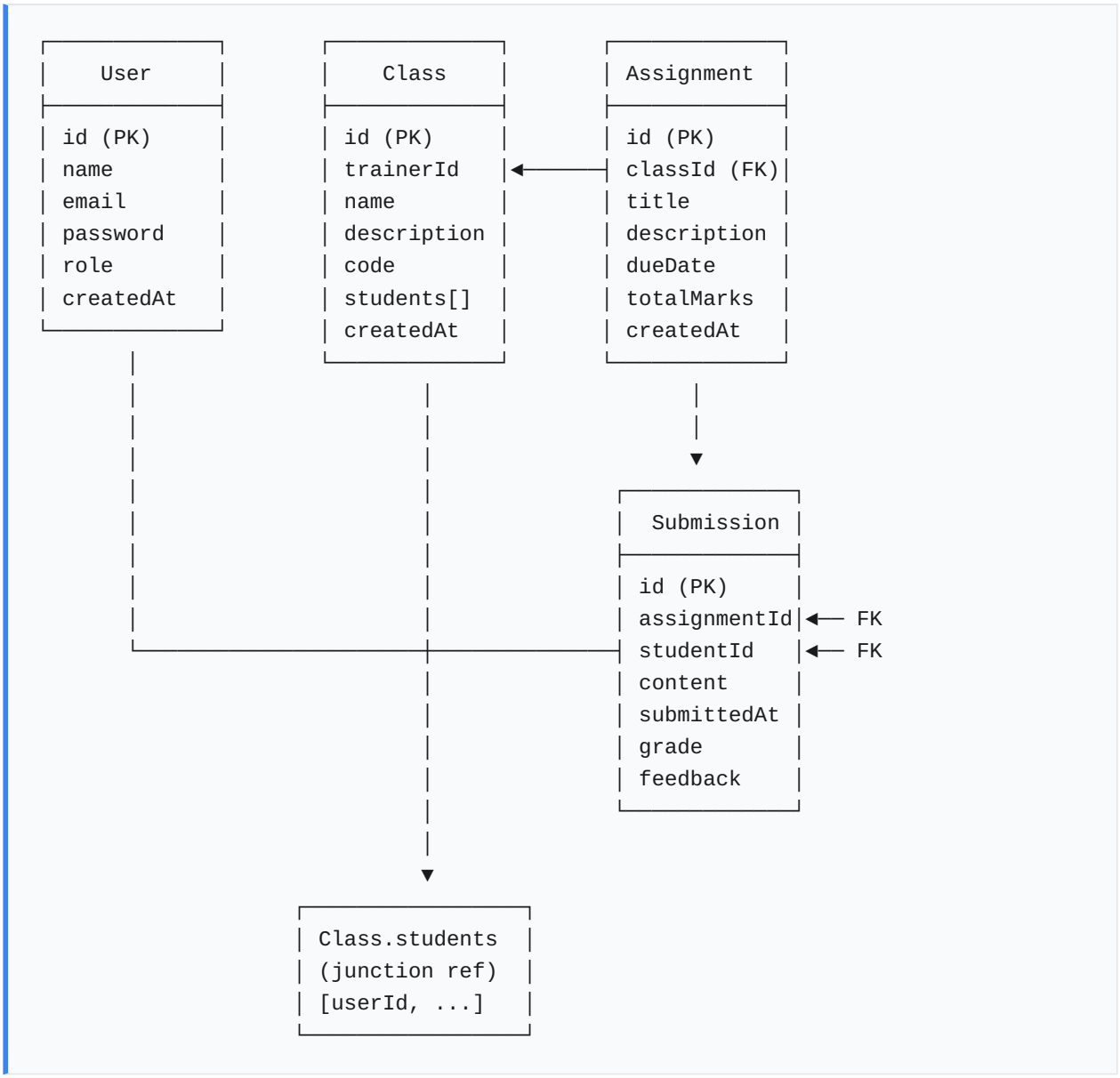
The sidebar items are defined in a configuration object keyed by role:

```
const sidebarConfig = {
    admin: [
        { id: 'overview', icon: 'fa-chart-pie', label: 'Overview' },
        { id: 'trainers', icon: 'fa-chalkboard-teacher', label: 'Trainers' },
        { id: 'students', icon: 'fa-user-graduate', label: 'Students' },
        { id: 'profile', icon: 'fa-user-cog', label: 'Profile' }
    ],
    trainer: [ /* 5 items */ ],
    student: [ /* 5 items */ ]
};
```

This declarative approach makes it trivial to add new pages or reorder navigation without touching rendering logic.

7. Data Models & Entity Relationships

7.1 Entity Relationship Diagram



7.2 Schema Definitions

User

```
{
    id: string,          // Unique identifier (base36 timestamp + random)
    name: string,         // Display name
    email: string,        // Login credential (unique)
    password: string,     // Plaintext (demo only — hash in production)
    role: 'admin' | 'trainer' | 'student',
    createdAt: string     // ISO 8601 timestamp
}
```

Class

```
{
    id: string,
    trainerId: string,    // FK → User.id (role must be 'trainer')
    name: string,
    description: string,
    code: string,         // 6-char uppercase join code
    students: string[],   // Array of User.id (students)
    createdAt: string
}
```

Assignment

```
{
    id: string,
    classId: string,      // FK → Class.id
    title: string,
    description: string,
    dueDate: string,      // YYYY-MM-DD format
    totalMarks: number,  // Positive integer
    createdAt: string
}
```

Submission

```
{
    id: string,
    assignmentId: string, // FK → Assignment.id
    studentId: string,    // FK → User.id (role must be 'student')
    content: string,      // Student's answer text
    submittedAt: string,  // ISO 8601 timestamp
    grade: number|null,   // Null until graded
    feedback: string|null   // Trainer's optional feedback
}
```

8. User Flows by Role

8.1 Admin Flow

ADMIN

```
Login → Overview (metrics)
  │
  ├─▶ Trainers ──▶ Add Trainer (modal form)
  │              ──▶ Delete Trainer (cascade: classes → assignments → submissions)
  │
  ├─▶ Students ──▶ Add Student (modal form)
  │              ──▶ Delete Student (cascade: submissions + class enrollments)
  │
  └─▶ Profile ──▶ Update name/email/password
```

8.2 Trainer Flow

TRAINER

```
Login → Dashboard (class count, assignment count, pending grading)
  │
  ├─▶ My Classes ──▶ Create Class (auto-generates join code)
  │              ──▶ Delete Class (cascade: assignments)
  │              ──▶ Manage Students (view enrolled + remove)
  │
  ├─▶ Assignments ─▶ Create Assignment (scoped to a class)
  │               ─▶ Delete Assignment (cascade: submissions)
  │
  ├─▶ Submissions ─▶ Review Pending → Grade (marks + feedback)
  │               ─▶ View Graded (edit existing grades)
  │
  └─▶ Profile ────▶ Update name/email/password
```

8.3 Student Flow

STUDENT

```
Login → Dashboard (enrolled classes, pending assignments, avg grade)
  │
  ├─► My Classes ──► Join Class (enter 6-char code)
  │              ──► View enrolled classes
  │
  ├─► My Assignments ─► Submit Work (text area)
  │                  ─► View Status (Pending / Submitted / Expired)
  │                  ─► View Grade (if already graded)
  │
  ├─► Grades ────────► Table: assignment, class, marks, %, feedback
  │              ─► Color-coded: green (≥80%), yellow (≥50%), red (<50%)
  │
  └─► Profile ───────► Update name/email/password
```

9. Deployment Guide

9.1 Local Development

Because the app uses ES6 modules (`type="module"`), you should serve it via a local server rather than opening `file://` directly (some browsers block module loading from file URLs).

Option A: Python HTTP Server

```
cd assignment-portal
python -m http.server 8000
# Open http://localhost:8000
```

Option B: Node.js (npx serve)

```
cd assignment-portal
npx serve
```

Option C: VS Code Live Server

Install the "Live Server" extension, right-click `index.html` , and select "Open with Live Server".

9.2 Production Deployment

GitHub Pages (Free)

1. Push the `assignment-portal` folder to a GitHub repository
2. Go to **Settings → Pages**
3. Select branch: `main` , folder: `/ (root)`
4. Site live at: `https://<username>.github.io/<repo>/`

Netlify (Free)

1. Visit netlify.com/drop
2. Drag and drop the `assignment-portal` folder
3. Instant live URL provided

Vercel (Free)

```
npm i -g vercel
cd assignment-portal
vercel
```

Traditional Hosting

Upload all files via FTP/SFTP. Ensure `index.html` is at the document root. No server-side processing (PHP, Node, etc.) is required.

9.3 Browser Support

Browser	Minimum Version	Notes
Chrome	80+	Full support
Firefox	75+	Full support
Safari	13.1+	Full support
Edge	80+	Full support

> ⚠ **Security Note:** Passwords are stored in plaintext in localStorage. This is acceptable for a demo/educational application but *must* be replaced with proper hashing (bcrypt/Argon2) and server-side authentication in any production system.

10. Quick API Reference

10.1 Store Methods

Method	Params	Returns	Side Effects
`init()`	—	void	Loads or seeds data
`save()`	—	void	Writes to localStorage
`login(email, password)`	2 strings	boolean	Sets currentUser + session
`logout()`	—	void	Clears currentUser + session
`generateId()`	—	string	None
`generateCode()`	—	string	None

10.2 Utils Methods

Method	Params	Description
`openModal(id)`	element ID	Shows modal by removing 'hidden'
`closeModal(id)`	element ID	Hides modal by adding 'hidden'
`showToast(msg, type)`	string, 'success'|'error'	Animated notification (3s)
`formatDate(str)`	ISO string	Locale date string

10.3 App Methods

Method	Params	Description
`navigate(page)`	page ID string	Routes to page, updates sidebar, animates content
`renderDashboard()`	—	Builds sidebar + main shell, navigates to overview
`handleAction(action, id, extra, event)`	4 params	Central dispatcher for all click events

10.4 Feature Module Entry Points

Module	Entry Method	Params
`Auth`	`render(app)`	App instance
`Admin`	`renderPage(page, container, app)`	page ID, DOM element, App instance
`Trainer`	`renderPage(page, container, app)`	page ID, DOM element, App instance
`Student`	`renderPage(page, container, app)`	page ID, DOM element, App instance
`Shared`	`renderProfile(container, app)`	DOM element, App instance

11. Troubleshooting

11.1 Common Issues

Symptom	Cause	Solution
Blank white screen	ES modules blocked by `file://` protocol	Use a local HTTP server (see Section 9.1)
Styles not loading	Tailwind CDN unreachable	Check internet connection or download Tailwind locally
Data resets on refresh	localStorage disabled or full	Enable localStorage in browser settings; clear old data
Cannot delete user	Trying to delete currently logged-in user	Log in as a different admin account first
Grade modal shows old data	Modal not cleared between openings	Click Cancel to close, then reopen
Mobile sidebar won't close	Click outside sidebar not handled	Click the hamburger button again to toggle

11.2 Resetting Application Data

To clear all data and return to the seeded defaults:

1. Open browser DevTools (F12)
2. Go to **Application → Local Storage**
3. Delete the keys: `portal_data` and `portal_session`
4. Refresh the page

11.3 Extending the Application

Adding a New Page

1. Add entry to `sidebarConfig` in `app.js` for the target role(s)
2. Add rendering case in the role's `renderPage()` method

3. Add any new `data-action` handlers to `App.handleAction()`
4. Add modal HTML inside the renderer if needed

Adding a New Entity

1. Add array to `store` object (e.g., `announcements: []`)
2. Update `store.save()` and `store.init()` to include the new array
3. Add CRUD methods in the appropriate feature module
4. Remember to call `store.save()` after every mutation

> ✓ **You're all set!** This documentation covers every file, every function, and every user flow in the Assignment Submission Portal. For questions or contributions, refer to the README.md included in the project root.